WebElement Control

Web element controls are the parts of a web page that a user can interact with, such as buttons, text boxes, checkboxes, radio buttons, dropdowns, and links. They are called 'web elements' because they are part of the page's HTML structure.In Playwright, we find them using locators and perform actions on them, like clicking or typing, just like a real user.

1)fill()
-->The fill() method is used to enter text or content into a text field or input field.
-->It takes the text that has to be entered in the field.
-->When we use fill(), it first removes any previous text present in the field and then adds the new content.
-->It takes the whole content passed as the argument and enters it into the field in one shot, instead of typing it character by character.
Syntax:

```
page.locator("").fill("me@example.com")
```

2)type()
-->the type() method is deprecated in Playwright. It was used to enter text or content into a text field or input field.
-->It takes the text that has to be entered in the field
-->When we use type(), it does not remove the previous text present in the field. It adds the new content after the existing text.
-->It takes the content passed as the argument and enters it character by character, like a real user typing on a keyboard, instead of entering it in one shot.
Syntax:

```
page.locator("").type("me@example.com")
```

3)click()
-->The click() method is used to click an element on the web page, such as a button, link, checkbox, or radio button.
-->It does not need any argument.
Syntax:

```
page.locator("").click()
```

4)clear()
-->the clear() method is used to clear the data that is present in a web element.
-->It does not need any argument.
Syntax:

```
page.locator("").clear()
```

5)innerText()

-->The innerText() method is used to capture the visible text present in a particular tag or web element, exactly as it appears to the user on the web page.
-->It does not need any argument.
-->It returns the visible text as a String.
-->It works on a single web element only. It cannot handle multiple web elements at a time.
-->It ignores hidden text. It returns only the text that is visible on the web page.
Example:

```
html
<div name="demo" type="element"> Button
    <span style="display:none"> Hello playwright </span>
</div>
Output: Button
The text "Hello playwright" is not returned because it is hidden (display:none).
```

Syntax:

```
text = page.locator("").innerText()
```

6)textContent()

-->The textContent() method is used to capture the complete text present in a web element, including both the visible text and the hidden text.
-->It does not need any argument.
-->Return value: It returns the full text as a String.
-->It works on a single web element only. It cannot handle multiple web elements at a time.
Example:

```
html
<div name="demo" type="element"> Button
    <span style="display:none"> Hello playwright </span>
</div>
Output: Button Hello playwright
The hidden text "Hello playwright" is returned as well, because text_content()
does not care whether the text is visible or hidden.
```

Syntax:

```
text = page.locator("").textContent()
```

7)inputValue()

-->The inputValue() method is used to capture the current text that is present in a text field or text area on the web page.
-->It does not need any argument.
-->Return value: It returns the data in String format
-->It is mainly used for <input> and <textarea> elements.
Syntax:

```
page.get_by_label("").fill("example")
let value = page.get_by_label("").inputValue()
print(value)  // example
```

8) allInnerTexts()
-->The allInnerTexts() method is used to capture the visible text of multiple web elements at a time. It handles all the elements that match a locator.
-->No argument.
-->Return value: An array of strings, one for each matching element, containing only the visible text. Hidden text is not included.

Syntax:

```
texts = page.locator("").allInnerTexts()
```

9)allTextContents()
-->The allTextContents() method is used to capture the visible text and hidden text of multiple web elements at a time. It handles all the elements that match a locator.
-->No argument.
-->Return value: An array of strings, one for each matching element, containing both the visible text and the hidden text.
Syntax:

```
texts = page.locator("").allTextContents()
```

10)getAttribute()
-->The getAttribute() method is used to get the value of a particular attribute of a located web element on the web page. For example, we can get the value of href, id, class, placeholder, or type.
-->The attribute name, passed in string format as an argument.
-->Return value: The attribute value in string format.

Syntax:

```
let ref = await page.locator("").getAttribute("attributeName");
```

11)all()
-->The all() method is used to retrieve all the matching web elements for a locator at a time on the web page and return An array of locators
--> No argument.
-->To go through the array, we can use a for loop.
-->It does not have auto-wait, so it works only for static applications.
-->If the elements are not present immediately, it does not wait and returns an empty array.

Syntax:

```
let elements = await page.locator("").all();
```

12)isVisible()
-->The isVisible() method is used to check or verify whether the located web element is visible on the web page or not.
-->No argument.
-->Return type: Boolean.
-->If the web element is visible, it returns true.
-->If the web element is not visible, it returns false.
-->It does not have auto-wait, so it does not wait for the element to appear. It gives the result immediately.

Syntax

```
let ref = await page.locator("").isVisible();
```

13)isEnabled()
-->The isEnabled() method is used to verify whether the located web element is enabled or not.
-->No argument.
-->Return type: Boolean.
-->If the web element is enabled, it returns true.
-->If the web element is not enabled (disabled), it returns false.
-->It does not have auto-wait, so it does not wait for the element. It gives the result immediately.

Syntax

```
let ref = await page.locator("").isEnabled();
```

14)isDisabled()
-->The isDisabled() method is used to verify whether the located web element is disabled or not.
-->No argument.
-->Return type: Boolean.
-->If the web element is disabled, it returns true.
-->If the web element is not disabled (enabled), it returns false.
-->It does not have auto-wait, so it does not wait for the element. It gives the result immediately.

Syntax

```
let ref = await page.locator("").isDisabled();
```

15)isEditable()
-->The isEditable() method is used to check whether the user can enter or change the data in a field.
-->No argument.
-->Return type: Boolean.
-->If the field accepts input (typing, editing, clearing), it returns true.
-->If the field is disabled or read-only, it returns false.
-->It is mainly used for <input> and <textarea> web elements.

Syntax:

```
let ref = await page.locator("").isEditable();
```

16)check()
-->The check() method is used to select radio button or a checkbox.
-->No argument.
-->Return type: void
-->Instead of the click() method, we can use check() to select a radio button or checkbox.
-->It works only on radio buttons and checkboxes.
-->If the element is already selected, it does nothing.
-->It has auto-wait, so Playwright waits until the element is visible, enabled, and stable before acting.

Syntax

```
await page.locator("#terms").check();
```

17)uncheck()
-->The uncheck() method is used to deselect a checkbox that is already selected.
-->No argument.
-->Return type: void (it performs an action and returns nothing).
-->We cannot use uncheck() on a radio button, because a radio button cannot be deselected directly.
-->To deselect a radio button, we select another radio button of the same group, and then the previously selected one is automatically deselected.
-->It has auto-wait, so Playwright waits until the element is visible, enabled, and stable before acting.

Syntax

```
await page.locator("#terms").uncheck();
```

18)isChecked()
-->The isChecked() method is used to verify whether a radio button or a checkbox is selected or not.
-->No argument.
-->Return type: Boolean.
-->If the element is selected, it returns true.
-->If the element is not selected, it returns false.
-->It does not have auto-wait, so it does not wait for the element. It gives the result immediately.

Syntax

```
let ref = await page.locator("#terms").isChecked();
```

19)waitFor()
-->The waitFor() method is used to pause the test execution until the located web element reaches a particular state. It ensures that the element is ready before any validation or action is performed.
-->It accepts argument with 2 optional values: state and timeout.
-->Return type: void (it returns nothing).
-->If the web element reaches the expected state, Playwright gives the control to the next line.
-->If the web element does not reach the expected state, Playwright does not move to the next line. It waits for the timeout (30 seconds by default), and then it throws an error
-->It has an auto-waiting mechanism.

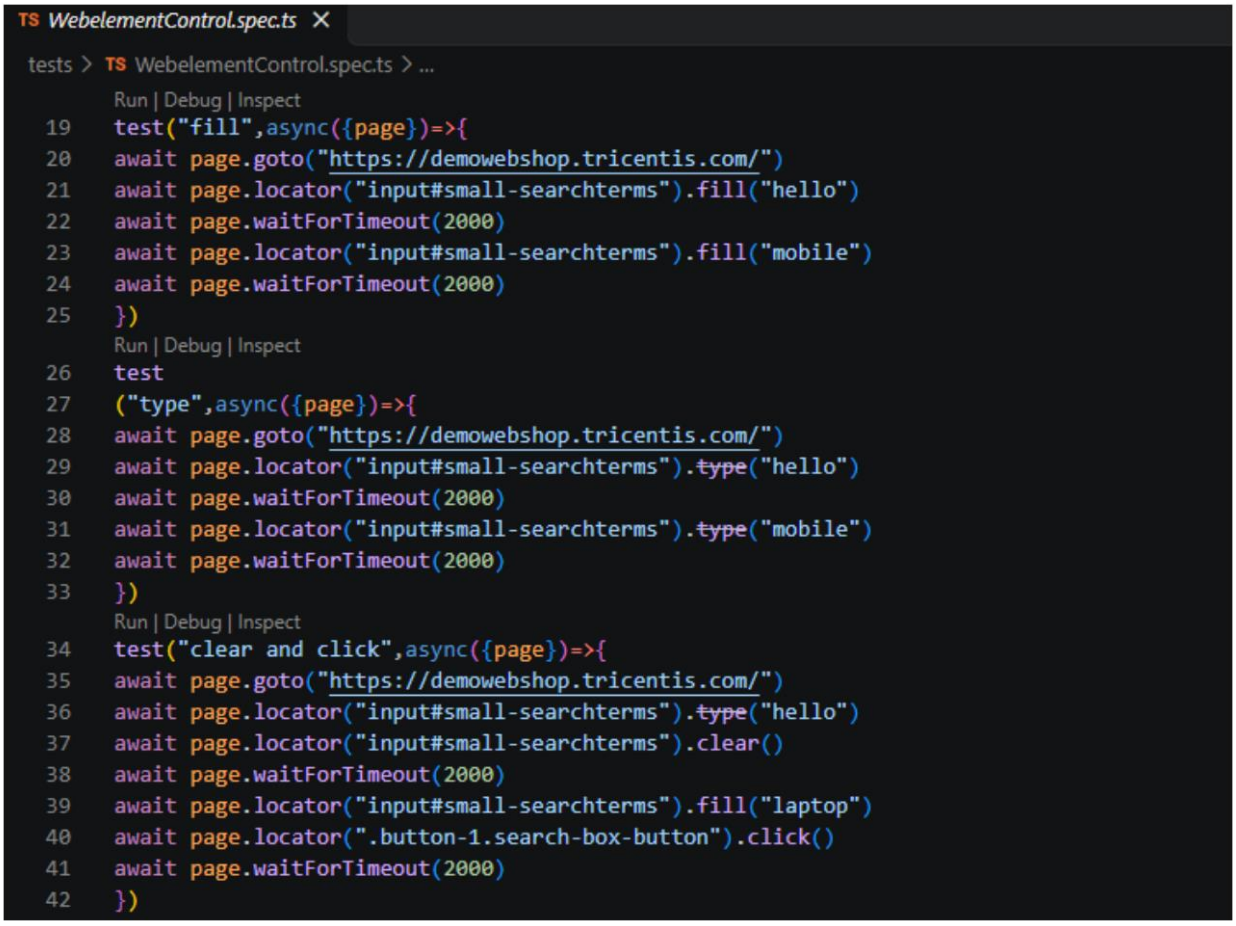
Syntax

```
await page.locator("").waitFor({ state: "visible", timeout: 10000 });
```

The 2 arguments:
state: the state we want to wait for.
timeout: the maximum time to wait, in milliseconds.

The 4 values of state:

State
visible=Waits until the element is attached to the DOM and visible on the web page. This is the default.
hidden=Waits until the element is not visible on the page. It may be present in the DOM but hidden, or removed from the DOM completely.
attached=Waits until the element is attached (present) in the DOM. It may or may not be visible.
detached=Waits until the element is removed from the DOM.



TS WebelementControl.spec.ts

tests > TS WebelementControl.spec.ts > ...

```ts
test("innerText",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
// let textofelement:string=await page.getByRole("link",
// {name:'$25 Virtual Gift Card',exact: true}).innerText()
// console.log(textofelement)
  let allproduct:Locator= page.locator(".product-title")
  console.log(allproduct)
//   console.log(await allproduct.nth(2).innerText())
   const count=await allproduct.count()
    for(let i=0;i<count;i++){
      console.log(await allproduct.nth(i).innerText())
    }
})
Run | Debug | Inspect
test("textcontext",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
// let textofelement:string=await page.getByRole("link",
// {name:'$25 Virtual Gift Card',exact: true}).textcontent()
// console.log(textofelement)
  let allproduct:Locator= page.locator(".product-title")
  console.log(allproduct)
//   console.log(await allproduct.nth(2).textcontent())
   const count=await allproduct.count()
    for(let i=0;i<count;i++){
      console.log(await allproduct.nth(i).textContent())
    }
})
test("inputvalue",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
let textfield= page.locator("input#small-searchterms")
await textfield.fill("hello")
console.log(await textfield.inputValue())
})
Run | Debug | Inspect
test("allinnerText",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
 let allproduct:string[]= await page.locator(".product-title").allInnerTexts()
 console.log(allproduct)
})
Run | Debug | Inspect
test("allStextcontent",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
 let allproduct:string[]= await page.locator(".product-title").allTextContents()
 console.log(allproduct)
})
```

```
test("all",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
//  let allproduct= page.locator(".product-title")
// console.log(allproduct)
let allproduct:Locator[]= await page.locator(".product-title").all()
for(let product of allproduct){
  console.log(await product.innerText())
}
// console.log(allproduct)
})
Run | Debug | Inspect
test("getAttribute",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
let textfield:Locator= page.locator("input#small-searchterms")
console.log(await textfield.getAttribute("type"))
console.log(await textfield.getAttribute("class"))
console.log(await textfield.getAttribute("id"))
console.log(await textfield.getAttribute("value"))
})
Run | Debug | Inspect
test("isvisble",async({page})=>{
await page.goto("https://www.saucedemo.com/")
await page.locator("#user-name").fill("standard_user")
await page.locator("[type='password']").fill("secret_sauce")
await page.locator(".submit-button").click()
let img:Locator= page.locator("[alt='Sauce Labs Backpack']")
await page.waitForTimeout(2000)
console.log(await img.isVisible())
})
```

```
113   test("isenabled",async({page})=>{
114   await page.goto("https://www.instagram.com/")
115   await page.locator("[name='email']").fill("amruta")
116   await page.locator("[name='pass']").fill("123")
117   await page.waitForTimeout(2000)
118   let button = page.locator("[aria-label='Log In']")
119   console.log(await button.isEnabled())
120   })
      Run | Debug | Inspect
121   test("isdisable",async({page})=>{
122   await page.goto("https://www.instagram.com/")
123   await page.locator("[name='email']").fill("amruta")
124   await page.locator("[name='pass']").fill("123")
125   await page.waitForTimeout(2000)
126   let button = page.locator("[aria-label='Log In']")
127   console.log(await button.isDisabled())
128   })
      Run | Debug | Inspect
129   test("iseditable",async({page})=>{
130   await page.goto("https://demowebshop.tricentis.com/")
131   let textfield:Locator= page.locator("input#small-searchterms")
132   console.log(await textfield.isEditable())
133   })
```

```
134  test("check and uncheck ",async({page})=>{
135  await page.goto("https://testautomationpractice.blogspot.com/")
136  //radio
137  await page.locator("#female").check()
138  await page.waitForTimeout(2000)
139  //checkboxes
140  await page.locator("#monday").check()
141  await page.locator("#tuesday").check()
142  await page.waitForTimeout(2000)
143  await page.locator("#monday").uncheck()
144  await page.locator("#tuesday").uncheck()
145  await page.waitForTimeout(2000)
146  })
     Run | Debug | Inspect
147  test("ischecked",async({page})=>{
148  await page.goto("https://testautomationpractice.blogspot.com/")
149  //radio
150  console.log(await page.locator("#female").isChecked())
151  await page.locator("#female").check()
152  console.log(await page.locator("#female").isChecked())
153  })
154   test.only("waitfor",async({page})=>{
155   await page.goto("https://www.saucedemo.com/")
156   await page.locator("#user-name").fill("standard_user")
157   await page.locator("[type='password']").fill("secret_sauce")
158   await page.locator(".submit-button").click()
159   let img:Locator= page.locator("[alt='Sauce Labs Backpack']")
160    await page.locator("[alt='Sauce Labs Backpack']").
161    waitFor({state:'visible',timeout:2000})
162   console.log(await img.isVisible())
163   })
```